C++ Programming

Juha Yli-Hemminki

04.12.2025

SeAMK

SeAMK

Templates

- Templates enable generic programming and code reuse
- Compiler generates type-specific versions
- Avoids repeating logic for multiple types
- Examples:
  - add<T> → T = int → generates add<int>
  - add<T> → T = double → generates add<double>

SeAMK

Function templates

- Define a function that works for any type
- Example:
  - template<typename T> T add(T a, T b);
- Compiler instantiates needed versions
- Type deduction simplifies usage:
  - add<int>(1, 2); // T = int
  - add(2.5, 4.1); // T = double

Class templates

- Generic classes that store/work with any type
- Used in containers (vector, map), smart pointers (unique_ptr), utilities (pair)
- Example:
  -
```
template<typename T> class Box {
  T value;
};
```

SeAMK

Multiple template parameters

- Templates can accept several type parameters
- Example:
  - template<typename T, typename U> class Pair;
- Supports combining multiple independent types

std::vector<T>

- Dynamic array where T determines actual stored type
- Internals:
  - T* data
  - size, capacity
  - allocator<T>
- vector<int> vs vector<string>

SeAMK

Templates summary

- Zero-cost abstraction: compiler optimizes per type
- Strong type safety
- Avoids code duplication
- Foundation of containers, smart pointers, and STL algorithms
- Compile-time calculation improves safety and performance
- Essential for modern C++ and generic programming

SeAMK

SEINÄJOEN AMMATTIKORKEAKOULU

SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES